Use Cases

1. **Load textures:** the developer configures and loads base or custom textures for towers, enemies, strategic assets, GUI, background (map) and projectiles.
2. **Level entities creation:** the developer creates entities (e.g. strategic assets, tower spots, enemies spots) by composing components from predefined set.
3. **Adding component to entity:** the developer opens a menu in entities editing window and chooses which component to add to the entity
4. **Connecting enemy path to strategic assets and spawn points.**
5. **Handle error:** the editor displays an error message.
6. **Configuring enemy spawn points:** the developer creates new entities and adds “spawn point” component to them, in this component the developer configures enemy waves: sets spawn rate and types of enemy entities spawn point can create during each wave.
7. **Configuring enemy targeting logic:** the developer adds one of available “tactic” components to the enemy entity and configures parameters of this component to adjust targeting logic.
8. **Damage customization:** the developer can add new types of damage (instant, ongoing, from some effects, etc.).
9. **Configuring enemy health:** the developer adds a “mortal” component to the enemy entity and configures the enemies’ health points amount (possibly of different types, like armor, physical health etc.).
10. **Tower customization:** the developer sets the tower type, target logic, projectiles collision, attack range, attack

cooldown, upgrade/sell system, active/inactive modes by configuring components of tower entities.

11. **Grid and map layout setup**: configuring tile-based grid properties, background assets and zones of the level map.
12. **Enemy path drawing:** defining and drawing the route trajectories and movement waypoints that enemies will follow from spawn points to the strategic asset.
13. **Entity placement and spawning**: selecting game entities (towers, enemy spots, strategic assets) from a palette and placing them onto the map.
14. **Entity duplication:** cloning an existing entity along with all its configured components and parameters.
15. **Group operations**: selecting multiple entities to move, duplicate or delete them in group.
16. **Level configuration generation:** the editor generates a level description as a configuration file of the format recognizable by the game engine executable.
17. **Game execution**: the game engine executable reads the generated configuration file to launch the game logic and rendering.
18. **Launch the game:** the user launches the game using the generated configuration file.
19. **Place a tower:** the player selects a tower type and places it in an available position on the map.
20. **Sell a tower:** the player sells a selected tower and receives part of its cost back.
21. **Upgrade a tower:** the player upgrades a selected tower using available resources.
22. **Use an ability:** the player activates an available special ability during the game.
23. **Start the next wave:** the player starts the next enemy wave.

24. **Change game speed:** the player increases or decreases the game speed
25. **Pause or resume the game:** the player pauses the current game session or resumes it after the pause.
26. **Restart the game:** the player restarts the current game from the beginning.
27. **Exit the game:** the user stops the current game session and closes the game.

Detailed Descriptions

1. Load textures

Actors: developer
Goal: configure and load base or custom textures for all game visuals.
Trigger condition: developer opens textures/assets panel in the editor.
Precondition/Context: editor is running; asset directory is available; supported formats known.
Extensions:

- (unsupported format or duplicate name, 3, Handle error)

Main success scenario:

1. Developer opens the texture panel in the editor;
2. Developer selects a category (tower, enemy, etc.);
3. Developer imports a base or custom texture file;
4. Editor validates format and assigns texture to the category;
5. Editor previews texture in contexts (on map or in list);
6. Developer saves, editor writes texture references into the level configuration.

Alternative scenarios:

- developer replaces an existing texture - old reference updated everywhere;
- developer removes a texture - falls back to default or marks slot empty.

2. Enemy path drawing

Actors: developer
Goal: create a path entity with a curve component and link and enemy spawn point and strategic asset to it
Trigger condition: the developer presses the button in the “enemy path” component editing menu to edit the “shape” component field of type “curve” and enters the special editor mode for curve drawing.
Precondition/Context: the level editor is running, the map layout is set up, enemy spawn points and strategic asset are already placed on the map, there is an entity with an “enemy path” component.
Main success scenario:

1. The developer places sequential waypoints on the grid to define the movement trajectory
2. When the arrangement is complete, the user clicks the “complete editing” button and exits the special editor mode.

3. Connecting enemy path to strategic assets and spawn points

Actors: developer
Goal: Connect enemy path to strategic assets and spawn points
Trigger condition: the developer presses the button in the “connected to the path” component editing menu to edit the “path reference” component field of type “reference to entity with enemy path component”.

Precondition/Context: the level editor is running, there is an entity with an “connected to the path” component.
Main success scenario:

1. Editor opens new window with list of entities that have “enemy path” component
2. The developer choose the appropriate path entity from this list
3. The window closes

4. Tower Customization

Actors: developer
Goal: define a tower type by composing and configuring its components: type, targeting logic, attack range and cooldown, upgrade system, selling price?
Trigger condition: the developer creates a new tower entity or opens an existing one in the tower editor panel.
Precondition/Context: the editor is running, a predefined component set is available, textures for the tower and its projectiles have been loaded via Texture Customization.
Main success scenario:

1. The developer opens the tower editor panel and creates a new tower entity (or selects an existing one);
2. The developer sets the tower type, assigns texture for the tower body and selects projectiles for firing;
3. The developer configures target logic (first, last, strongest, weakest, closest);
4. The developer configures attack range and cooldown;
5. The developer configures upgrading tree(?);
6. The developer previews the tower in context (range circle, UI card);
7. The developer saves => the editor serializes the tower entity and all its components into the level configuration.